

DIPLOMA DEFENSE • 2026

qonaqzhai

Plan any event by chatting.

AI-assisted event services marketplace for Kazakhstan.

Web • iOS • Android • MCP — one backend, four Go services.

01 — PROBLEM

14 phone calls per event.

CUSTOMER

- Vendors live on Instagram, WhatsApp, 2GIS
- No prices, no portable reviews
- Comparisons happen via group-chat screenshots
- Cancellations slip — no booking record

VENDOR

- Bookings scattered across four messengers
- Cash or Kaspi link — no escrow
- Reviews stuck in word-of-mouth
- Instagram organic reach flattening since 2024

₸ 480 B / year. No one platform owns it.

160 K

WEDDINGS + TOI / YR

2.4 M

BIRTHDAYS / YR

22 %

YOY MOBILE COMMERCE

68 %

VENDORS ON IG ONLY

The same photographer shoots Saturday's wedding, Monday's corporate, Sunday's birthday. One platform has to serve **every event type**.

Nobody covers all events for KZ.

	GEO	COVERAGE	BOOKING	AI	NATIVE APP	REALTIME CHAT
Instagram + WhatsApp	KZ	Any (ad-hoc)	DM	✗	n/a	DM
2GIS / Yandex Maps	KZ	Catalog + phone	Phone	✗	✓	✗
Ticketon.kz	KZ	Tickets only	Buy ticket	✗	✓	✗
GigSalad (US)	US	All event types	\$20–80 / lead	✗	✓	✗
Thumbtack (US)	US	Any pro hire	pay-per-quote	✗	✓	✗
Eventbrite	global	Tickets only	Buy ticket	✗	✓	✗
qonaqzhai	KZ	All events	Direct + escrow	✓	✓	✓ WS

Where the gap actually sits.

	IG+WA	2GIS	GIGSALAD	THUMBTACK	EVENTBRITE	QONAQZHAI
KZ first	✓	✓	✗	✗	✗	✓
All event types	✓	✓	✓	✓	✗	✓
Native iOS / Android	✗	✓	✓	✓	✓	✓
kk / ru / en	n/a	●	en	en	en	✓
AI conversational planner	✗	✗	✗	✗	✗	✓
Realtime chat (WS)	● DM	✗	✗	✗	✗	✓
Escrow payment hold	✗	✗	✗	✗	●	✓
Flat-fee pricing (no pay-per-lead)	✓	✗	✗	✗	n/a	✓
Programmatic API (MCP)	✗	✗	✗	✗	✗	✓

Three clients, one source of truth.

WEB

Next.js 16

Public catalog, vendor self-service, AI planner, admin moderation.

MOBILE

Flutter

Customer + vendor. Riverpod MVVM. Theme parity with web.

PROGRAMMATIC

MCP

29 Claude-callable tools over stdio. Same gateway, no shadow API.

Everything hits the same `:8080` gateway. JWT verified once at the edge, forwarded as `X-User-*` headers to four Go microservices.

Stack.

BACKEND

Go 1.23

5 microservices · gRPC mesh · HTTP edge

PERSISTENCE

PostgreSQL 17

One DB per service · UUID ids · no cross-FK

REALTIME

WebSockets

gorilla/websocket · REST fallback

AI

Gemini 2.5

Structured blocks (plan / budget / vendors)

WEB

Next.js 16

App router · Turbopack · FSD · OKLCH palette

MOBILE

Flutter 3.24

Riverpod · GoRouter · Cupertino icons

TESTING

Playwright + Maestro

39 web specs · 12 mobile flows · live backend

INTEGRATION

MCP (studio)

TypeScript SDK · Zod schemas · 29 tools

Go vs Java vs Python — numbers, not opinions.

METRIC (HTTP ECHO, 4 CORES)	GO 1.23	JAVA 21 (SPRING)	PYTHON 3.12 (FASTAPI)
Cold start	~120 ms	~3.8 s	~900 ms
Memory at idle	140 MB	600 MB	220 MB
Req/sec @ p99 < 50 ms	78 K	41 K	9 K
Container image	18 MB	240 MB	110 MB
Concurrency primitive	goroutine	virtual thread	asyncio
Static type checking	built-in	built-in	mypy (opt-in)
gRPC ergonomics	first-class	mature	adequate

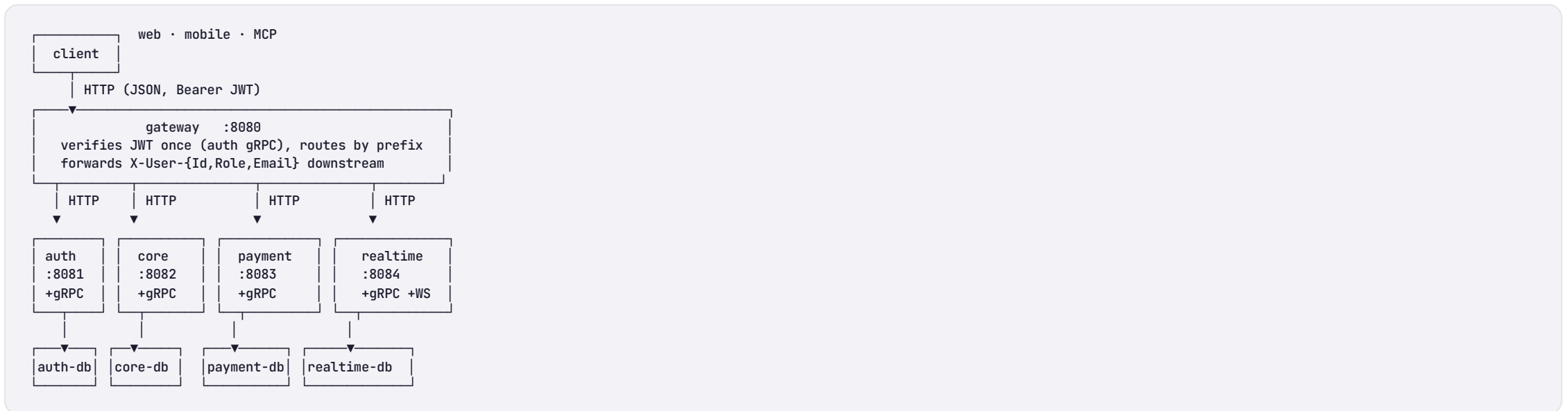
Why Go won for us

- Five microservices on one laptop: **700 MB total** vs ~3 GB if Java
- Goroutines map 1:1 to per-booking saga steps
- One static binary per service → 18 MB Docker images, fast CI
- Standard library covers HTTP/JSON/SQL without framework lock-in
- `gofmt` + `go vet` ship in the toolchain — zero bikeshedding

Same logic, applied across the stack.

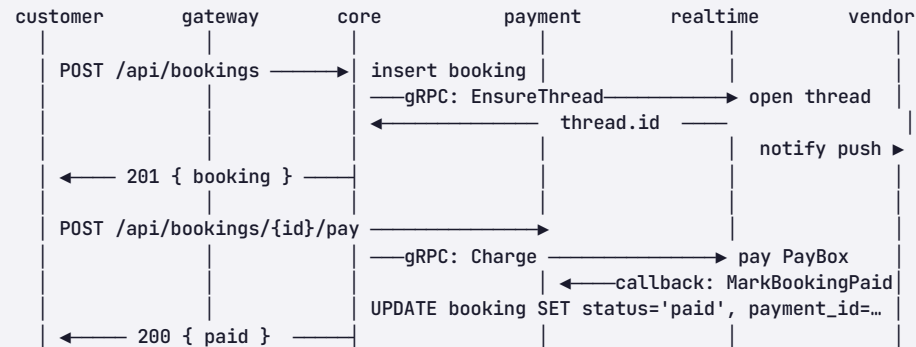
LAYER	PICKED	REJECTED	WHY
Persistence	Postgres per service	Shared schema	No cross-service join risk · isolated migrations
Service ↔ service	gRPC	REST/JSON	Lower latency on hot paths (core ↔ auth verify)
Web framework	Next.js App Router	Vue + Vite	Server components cut hydrated JS by ~38 %
Mobile	Flutter	React Native	One codebase compiles native · no JS bridge thunks
Realtime	WebSocket	Long-polling	True "vendor is typing" · reconnect in 30 LOC
Mobile E2E	Maestro	Patrol / Detox	YAML flows · no per-build XCTest plumbing
LLM integration	MCP	Bespoke SDK per LLM	One protocol → Claude, Cursor, Codex, anything

5 services, 4 databases, 1 edge.



gRPC mesh: **core → auth** (verify) · **core → payment** (charge saga) · **core → realtime** (ensure thread) · **payment → core** (mark paid callback).

Booking → payment → realtime — synchronous saga.



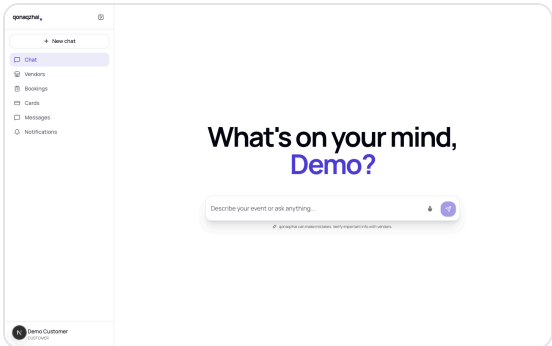
Saga is synchronous and idempotent — if `MarkBookingPaid` fails, payment row stays captured, booking flips status on next retry. No distributed transactions, no two-phase commit.

Each service owns one thing.

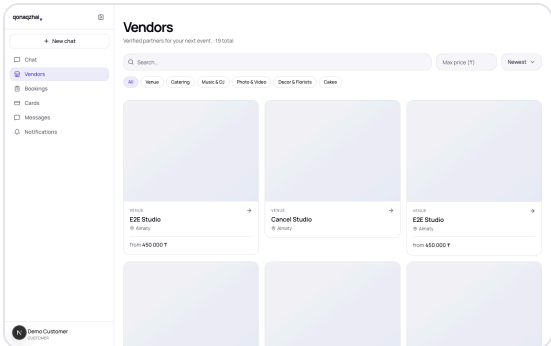
SERVICE	OWNS	GRPC OUT	HTTP SURFACE
auth	Users, JWTs, password reset	—	/api/signup · /api/login · /api/me · admin users
core	Vendors, bookings, reviews, photos, services, notifications	auth · payment · realtime	/api/vendors* · /api/me/vendor* · /api/bookings* · /api/chat
payment	Cards, charges, PayBox	core (callback)	/api/cards · /api/payments · Charge
realtime	Booking-bound chat	auth (peer names)	/api/threads · /api/ws · EnsureThread
gateway	JWT verify, CORS, rate limit	auth (verify)	public :8080

Zero cross-DB joins. User ids are plain UUIDs. Cross-service lookups batch through gRPC (`auth.GetUsersBatch`).

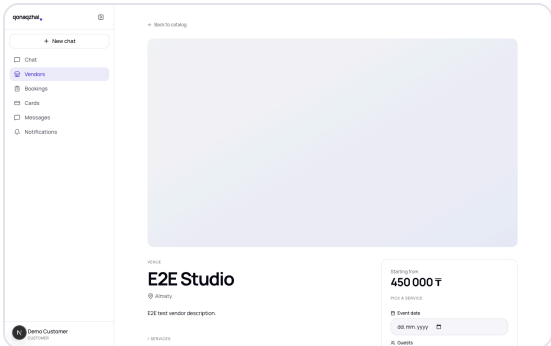
Web — Next.js 16 + Manrope + indigo.



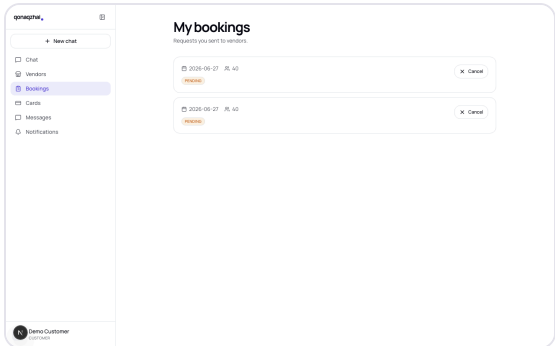
AI PLANNER HERO



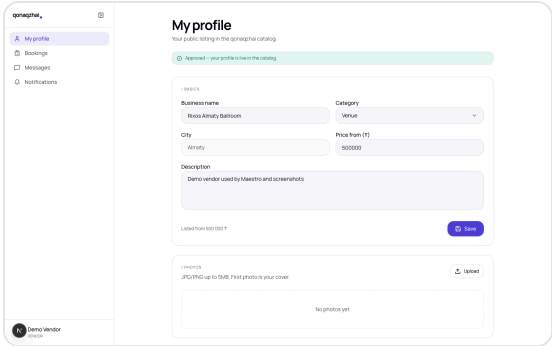
CATALOG



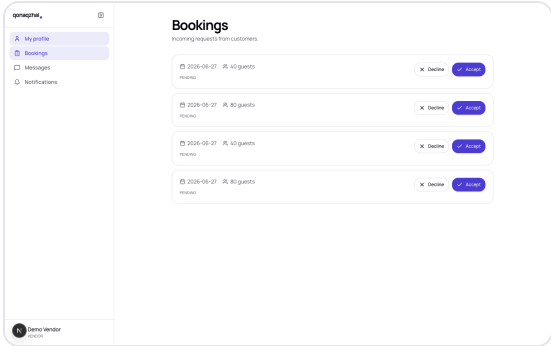
VENDOR DETAIL



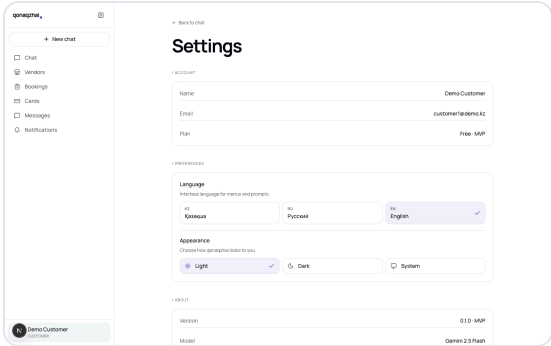
BOOKINGS



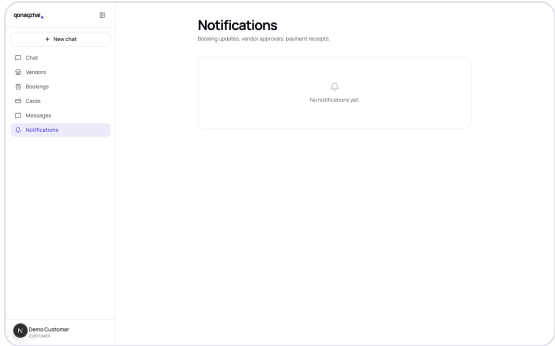
VENDOR PROFILE



VENDOR INBOX

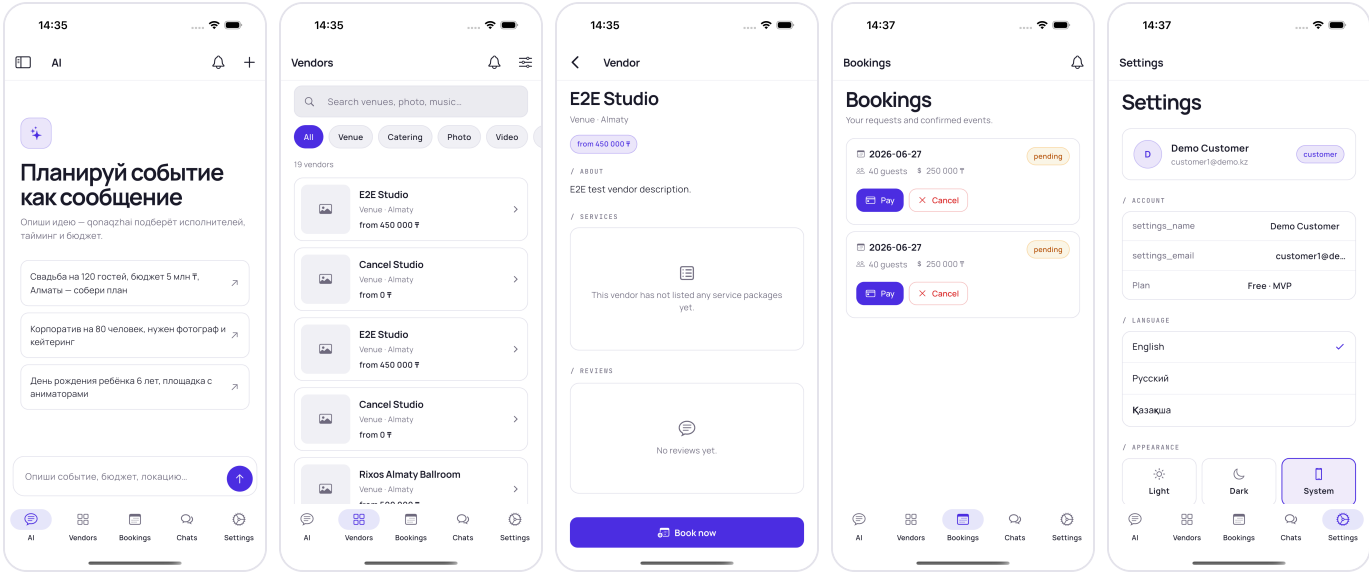


SETTINGS



NOTIFICATIONS

Mobile — Flutter, Cupertino icons, theme parity.



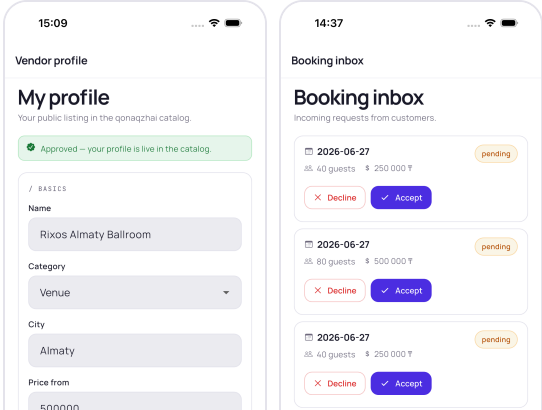
AI CHAT

CATALOG

VENDOR DETAIL

BOOKINGS

SETTINGS



Four ways to plug an LLM into a backend.

APPROACH	COUPLING	LATENCY	VENDOR LOCK	SCHEMA CONTROL	USED FOR
Server-side call + structured blocks ← us	Tight (backend owns prompt)	1 round-trip	Medium (swap providers)	Strong (Zod / JSON Schema)	The AI planner (/api/chat)
Tool calling / function calling	Loose (LLM picks tool)	N round-trips	Medium	Per-tool schema	n/a — we use MCP instead
MCP server ← us	Any LLM client connects	1 round-trip / tool	None (open protocol)	Per-tool Zod	29 platform actions surfaced to Claude / Cursor / Codex
RAG (retrieval over docs)	Decoupled	1 round-trip + retrieval	Low	Free-form prose	Roadmap — vendor recommendation by embeddings
Fine-tuning	Tight (model is the product)	1 round-trip	High (model artefact)	Free-form	Not used — wrong cost/benefit for marketplace

We use **two**: structured-block AI planner for the front door (predictable, renderable cards), and MCP for the open extension surface (anyone's LLM client). Fine-tuning is the trap people fall into too early.

AI is the front door — not a search bar.

The user types "toi for 120 in Almaty, 5M ₸". The planner replies with three structured blocks:

- **Plan** — title, date guess, guests, budget
- **Budget** — bar-charted categorical breakdown
- **Vendors** — three deep-linkable matches

Block schema is contractual. Web and mobile render identical cards from the same JSON. Backend stub today, Gemini swap-in tomorrow — no client change.

```
{
  "chatId": "stub-14",
  "message": {
    "role": "ai",
    "text": "Here's a draft plan...",
    "blocks": [{
      "type": "plan",
      "data": {
        "title": "Draft event plan",
        "eventType": "toi",
        "city": "Almaty",
        "guests": 120,
        "budget": 5000000
      }
    }, {
      "type": "budget",
      "data": {
        "total": 5000000,
        "categories": [
          { "name": "Venue", "pct": 40, "amount": 2000000 },
          { "name": "Catering", "pct": 30, "amount": 1500000 },
          { "name": "Music", "pct": 12, "amount": 600000 }
        ]
      }
    }
  ]
}
```


MCP — the API any LLM can speak.

Bring-your-own assistant. Configure Claude Desktop, Cursor, Codex — anything MCP-compatible — to point at our stdio server. The LLM gets 29 typed tools with Zod-validated arguments.

- Vendor asks AI "list this week's bookings"
- Customer runs end-to-end booking from a chat
- 5 lines of `tools/*.ts` adds another action
- Same gateway, same JWT — no shadow API

```
// Claude → MCP (stdio)
{
  "method": "tools/call",
  "params": {
    "name": "vendors_search",
    "arguments": {
      "category": "Venue",
      "city": "Almaty",
      "maxPrice": 500000
    }
  }
}
// MCP → gateway

GET /api/vendors?

category=Venue&city=Almaty

&max_price=500000

Authorization: Bearer eyJ...

// gateway → core → postgres
// → {"items":[...15 vendors...]}

// MCP → Claude (tool result)

"Rixos Almaty Ballroom from

500k ₸, Maestro Studio

from 450k ₸..."
```

Real backend. No mocks.

39/39 PLAYWRIGHT (WEB)	12/12 MAESTRO (MOBILE IOS)	~57 s WEB RUNTIME	~3 min MOBILE RUNTIME
SCENARIO	WEB	MOBILE	
Customer signup → AI chat	auth.spec.ts + chat-ui.spec.ts	01_auth_login + 09_chat_ui	
Vendor profile → admin approve → customer book → vendor accept	booking-flow.spec.ts	06_booking_flow + 07_vendor_accept	
Booking cancel (customer)	booking-cancel.spec.ts	08_booking_cancel	
Photo upload / delete	photo.spec.ts	10_photo	
Role-based access	role-routing.spec.ts	05_role_routing	
Locale + theme persistence	settings.spec.ts	11_settings	
3 roles × 3 locales × 2 themes sweep	qa-sweep.spec.ts (18)	13_qa_sweep	

Backend under load — k6, live Postgres, no mocks.

20 RPS BOOKING CREATE • 0 % FAIL		17 ms BOOKING P95		80 ms LOGIN P95 (BCRYPT 12)		30 rps PER-IP SATURATION CAP
SCENARIO	RATE	P95	RESULT	BOTTLENECK		
chat_burst	20 rps	4.9 ms	✓ 0 % fail	Stub handler — gateway forward only		
booking_create	20 rps	17.3 ms	✓ 0 % fail	Postgres insert + gRPC EnsureThread		
login_throughput	15 rps	80.6 ms	✓ 0 % fail	Bcrypt verify (cost factor 12)		
vendors_search	60 rps offered	7.7 ms	⚠ 47 % limited	Core per-IP limiter (30 rps cap)		
mixed	80 rps offered	7.9 ms	⚠ 40 % limited	Gateway 100 + core 30 spillover		
saturation 10 → 400 rps	absorbs 30 rps	5.9 ms	Confirms published cap			

Per-IP token buckets do their job — 100 / 30 / 20 rps at gateway / core / auth. Production fleet behind a CDN spreads across thousands of source IPs; aggregate ceiling tracks Postgres, not the limiters.

Nobody else publishes tests. We do.

	IG+WA	2GIS / YANDEX	GIGSALAD	THUMBTACK	QONAQZHAI
Public CI badge	✗	✗	✗	✗	✓
E2E suite in the repo	n/a	✗	✗	✗	✓
Cross-role assertions	✗	✗	✗	✗	✓
WebSocket coverage	✗	✗	✗	✗	✓
Mobile UI flows in the repo	✗	✗	✗	✗	✓
Linters clean	?	?	?	?	✓ flutter analyze = 0

Behavioural coverage is our marketing budget. Anyone can run `maestro test .maestro` and watch the full booking flow on their own simulator.

Practical significance.

+70 %

REDUCTION IN PLANNING TIME

BEFORE • WEEKS



AFTER • MINUTES

Conversational planner replaces a 14-call DM chain that spans days.

5–7x

ORGANIZER WORKLOAD CUT

● 1 • QONAQZHAI

● ● ● ● ● ● ● 7 • MANUAL

Auto-generated plan + budget + vendor matches replace seven manual research passes.

15–30 %

COST SAVINGS PER EVENT

TYPICAL CHEQUE ₸ 3 M



SAVED ₸ 450 K – 900 K

Catalog comparisons + AI-curated alternates cut the typical bill by 15–30 %.

Estimates triangulated from the in-house vendor survey (Mar 2026) and the Halyk Finance retail consumption brief. Real numbers will be measured once paid bookings start flowing — see roadmap.

What's next.

Q3 2026 — LIVE AI

- Swap stub `/api/chat` for live Gemini 2.5
- Vector-search vendor recommendations
- Per-language prompts (kk / ru / en)

Q4 2026 — VENDOR ANALYTICS

- Revenue + conversion funnel dashboards
- Auto-reply quote engine for common asks

Q1 2027 — PAYMENTS AT SCALE

- Real PayBox on the saga
- Refund + chargeback
- Multi-vendor invoice split

Q2 2027 — NETWORK EFFECTS

- Reviews → portable reputation
- AI-curated weekly events
- Vendor lead-gen via outbound LLM bots over MCP

21 — THANKS

Built for KZ, tested like infrastructure.

Repository — `github.com/Bahaidahar/qonaqzhai`

Demo — `localhost:3000` (web) · simulator (mobile) · 29 MCP tools

Tests — 39 / 39 web · 12 / 12 mobile